

Analisa Kelemahan *Website* Pemerintah Kota XYZ Menggunakan Metode *Penetration Testing Execution Standard* (PTES)

Afryan Fanindya Prayogi
Fakultas Informatika
Telkom University
Bandung, Indonesia

afryanprayy@student.telkomuniversity.ac.id

Muhamad Irsan
Fakultas Informatika
Telkom University
Bandung, Indonesia

irsanfaiz@telkomuniversity.ac.id

Abstrak — *Website* pemerintah Kota XYZ menyediakan layanan informasi publik dan administrasi, namun keterbukaan akses meningkatkan risiko serangan siber. Penelitian ini bertujuan menganalisis kelemahan keamanan *website* tersebut menggunakan pendekatan *black box* dengan alur *Penetration Testing Execution Standard* (PTES) dari *Pre-engagement Interactions* hingga *Reporting*. *Intelligence Gathering* dilakukan melalui identifikasi teknologi dan resolusi domain, kemudian *Vulnerability Analysis* dilakukan pada konfigurasi server dan komponen WordPress menggunakan pemindaian terarah. Tahap eksploitasi dilakukan secara terkendali meliputi uji SQL injection, uji *flood* untuk menilai ketahanan layanan, uji *brute force* langsung terhadap *website*, dan uji *clickjacking* berbasis *iframe*, tanpa tindakan destruktif dan dengan menjaga ketersediaan layanan. Hasil menunjukkan SQL injection tidak terkonfirmasi, uji *flood* menyebabkan alamat penguji dibatasi oleh sistem, tetapi layanan tetap dapat diakses secara normal oleh pengguna lain, dan *brute force* tidak menghasilkan sesi tidak sah serta menunjukkan pembatasan percobaan. Temuan paling signifikan adalah *clickjacking* berhasil karena kontrol anti *framing* belum efektif, sehingga disarankan penerapan *header anti framing*, penguatan kontrol autentikasi, dan evaluasi ulang setelah perbaikan.

Kata kunci — Keamanan Siber, *Penetration Testing*, *Website*, PTES, WordPress

I. PENDAHULUAN

Seiring dengan berkembangnya teknologi informasi, dalam satu dekade terakhir terdapat dampak yang signifikan terhadap cara instansi pemerintahan menjalankan tugas dan memberikan pelayanan kepada masyarakat. Di tengah tuntutan efisiensi, transparansi, dan keterbukaan informasi publik, pemerintah dituntut untuk terus beradaptasi dengan kemajuan teknologi digital [1]. Salah satu bentuk dari digitalisasi tersebut adalah penggunaan *website* resmi sebagai media utama untuk menyampaikan informasi, menerima aduan masyarakat, menyediakan layanan administratif, hingga menjadi sarana promosi program-program strategis daerah.

Namun di balik bentuk-bentuk tersebut, terdapat tanggung jawab besar dalam hal keamanan sistem. *Website* pemerintah bersifat publik dan menyimpan data yang krusial, menyebabkannya dijadikan target potensial bagi berbagai serangan siber. Dalam beberapa kasus, kelemahan pada *website* instansi pemerintah dimanfaatkan oleh pihak tidak bertanggung jawab untuk melakukan tindakan seperti penyisipan *malware*, termasuk *trojan* dan *backdoor*, pencurian informasi, penghapusan data, hingga serangan yang dapat mengganggu layanan dan sistem secara signifikan [2]. Tidak sedikit pula serangan yang dilakukan hanya untuk menunjukkan eksistensi kelompok tertentu (*hacktivism*) atau sebagai bentuk protes digital terhadap kebijakan pemerintahan [3]. Hal ini menunjukkan bahwa keberadaan *website* tanpa perlindungan yang memadai akan menjadi titik masuk yang membahayakan stabilitas informasi dan citra institusi.

Tantangan keamanan ini diperburuk dengan fakta bahwa masih banyak pengelola sistem informasi di lingkungan pemerintah daerah yang belum menjadikan pengujian keamanan sebagai bagian dari proses rutin. *Website* sering dibuat dan dijalankan tanpa pengujian keamanan mendalam dan umumnya hanya diuji fungsionalitasnya saja. Akibatnya, potensi kerentanan tidak teridentifikasi sejak awal, sehingga membuka peluang serangan yang dapat terjadi kapan saja. Untuk mengantisipasi hal ini, perlu adanya pendekatan yang sistematis dalam menguji keamanan *website* dengan mengevaluasi kesiapan sistem dalam menghadapi potensi ancaman siber [4].

Salah satu metode yang dapat digunakan untuk mengatasi permasalahan ini adalah *Penetration Testing Execution Standard* (PTES). PTES merupakan standar metodologi yang digunakan dalam pelaksanaan pengujian penetrasi secara profesional dan telah diadopsi oleh banyak praktisi keamanan siber di seluruh dunia. Metode ini memiliki alur kerja yang komprehensif, mulai dari tahap *pre-engagement interaction*, *intelligence gathering*, *threat modeling*, *vulnerability analysis*, *exploitation*, *post exploitation*, hingga *reporting* [5]. Dengan adanya tahapan yang jelas tersebut, PTES akan memberikan jaminan bahwa

proses pengujian berjalan secara objektif, sistematis, dan dapat dipertanggungjawabkan.

Website milik pemerintah Kota XYZ berfungsi sebagai portal informasi publik dan juga memiliki sejumlah fitur layanan berbasis web. Keberadaan fitur-fitur tersebut melibatkan komponen seperti *form input*, penyimpanan data, serta koneksi ke *server* internal. Berdasarkan informasi awal dari pihak pengelola, *website* tersebut pernah dilaporkan mengalami perubahan tampilan yang tidak sesuai dengan kondisi normal yang mengarah pada dugaan *defacement*, serta sempat mengalami gangguan ketersediaan layanan sehingga tidak dapat diakses pada periode tertentu. Informasi ini masih memerlukan penguatan melalui verifikasi teknis dan dokumentasi insiden, namun kondisi tersebut menunjukkan adanya risiko nyata terhadap layanan publik apabila pengujian keamanan tidak dilakukan secara terstruktur. Hingga saat ini belum terdapat pengujian mendalam terhadap aspek keamanan *website* dalam kerangka kerja standar industri seperti *Penetration Testing Execution Standard* (PTES), sehingga perlu dilakukan evaluasi sistematis untuk menilai potensi kerentanan dan menyusun rekomendasi perbaikan yang relevan.

Berdasarkan latar belakang tersebut, penelitian ini menggunakan perspektif *black box* dengan melakukan observasi terhadap input dan output sistem tanpa memiliki akses ke *source code*, konfigurasi internal, maupun *execution path*. Objek penelitian adalah *website* pemerintah daerah XYZ, dengan proses *assessment* yang mengikuti *Penetration Testing Execution Standard* (PTES) untuk menyusun seluruh aktivitas mulai dari tahap *pre-engagement* hingga *reporting*, sehingga analisis *vulnerability* didasarkan pada perilaku sistem yang dapat diamati dari sisi eksternal.

II. KAJIAN TEORI

A. Penetration Testing

Penetration testing merupakan salah satu bentuk *security testing* di mana seorang assessor mensimulasikan serangan nyata untuk menemukan cara melewati mekanisme proteksi pada sebuah aplikasi, sistem, atau jaringan [6]. Tujuan dari *penetration testing* adalah untuk mengungkap kelemahan yang berpotensi dieksploitasi oleh *attacker* serta menyediakan bukti teknis yang dapat digunakan sebagai dasar dalam peningkatan dan perbaikan kontrol keamanan.

TABEL 1
Pendekatan *Penetration Testing*

Pendekatan	Tujuan
<i>Black Box</i>	Pendekatan yang mensimulasikan serangan tanpa pengetahuan awal mengenai target, dengan tujuan menemukan <i>vulnerability</i> menggunakan teknik nyata seperti <i>social engineering</i> dan <i>remote access</i> .
<i>White Box</i>	Pendekatan pengujian di mana tester memiliki pengetahuan penuh terhadap infrastruktur sistem, sehingga penilaian risiko dilakukan berdasarkan informasi internal yang setara dengan level <i>inside</i> .
<i>Gray Box</i>	Pendekatan gabungan antara <i>black box</i> dan <i>white box</i> , di mana tester hanya memiliki informasi terbatas untuk

	mengidentifikasi permasalahan keamanan baik dari sisi internal maupun eksternal sistem [7].
--	---

B. *Penetration Testing Execution Standard* (PTES)

Sebuah metodologi *penetration testing* yang digunakan untuk mengorganisasi seluruh proses pengujian, mulai dari tahap perencanaan awal hingga pelaporan akhir, sehingga pengujian dapat dilakukan secara terstruktur, komprehensif, dan dapat dipertanggungjawabkan [8].

Tahapan dalam PTES terdiri dari beberapa fase sebagai berikut.

1. *Pre-engagement interactions*

Pada tahap *pre-engagement interactions*, tester dan pihak organisasi melakukan kesepakatan terkait ruang lingkup pengujian, tujuan, jenis pengujian yang dilakukan, batasan, aspek legal, serta mekanisme dokumentasi dan reporting.

2. *Intelligence gathering*

Tahap *intelligence gathering* merupakan proses pengumpulan informasi terkait target pengujian, seperti domain, IP address, serta data publik lainnya.

3. *Threat modelling*

Pada tahap *threat modeling*, pendekatan pengujian yang diperlukan ditentukan dengan memetakan aset penting, jalur serangan yang memungkinkan, serta potensi *adversary*.

4. *Vulnerability analysis*

Tahap *vulnerability analysis* bertujuan untuk mengidentifikasi kelemahan pada konfigurasi sistem dan aplikasi menggunakan teknik otomatisasi dan manual.

5. *Exploitation*

Pada tahap *exploitation*, pengujian difokuskan pada pembuktian secara terkontrol bahwa kelemahan yang telah diidentifikasi dapat disalahgunakan untuk mencapai tujuan *attacker* yang signifikan, tanpa menimbulkan kerusakan atau gangguan terhadap lingkungan target.

6. *Post-exploitation*

Tahap *post-exploitation* dilakukan untuk menilai sejauh mana dampak yang dapat terjadi setelah keberhasilan eksploitasi awal.

7. *Reporting*

Pada tahap *reporting*, seluruh proses pengujian dan bukti temuan didokumentasikan secara sistematis, disertai dengan penilaian risiko dan rekomendasi yang diprioritaskan.

C. Keamanan Jaringan

Keamanan Jaringan merupakan kombinasi antara mekanisme perlindungan teknis dan praktik manajemen yang diterapkan untuk melindungi traffic jaringan serta aset yang terhubung di dalamnya. Teknik yang umum digunakan dalam keamanan jaringan meliputi penerapan firewall untuk membatasi traffic berbahaya, penggunaan SSL atau TLS untuk melindungi data yang dikirimkan (data in transit), serta penerapan Intrusion Detection System (IDS) atau Intrusion Prevention System (IPS) untuk mendeteksi dan mencegah aktivitas yang mencurigakan. Selain aspek teknis, kebijakan keamanan, pembatasan akses, serta peningkatan user awareness juga memiliki peran yang signifikan dalam mencegah kesalahan yang dapat berujung pada terjadinya insiden keamanan [9].

D. Kali Linux

Kali Linux merupakan sistem operasi berbasis Debian yang dirancang khusus untuk keperluan *security testing* dan *digital forensics*. Sistem operasi ini menyediakan ratusan *open-source tools* yang umum digunakan oleh praktisi keamanan, serta mendukung instalasi penuh maupun mode *live boot* melalui USB sehingga dapat digunakan pada berbagai skenario laboratorium maupun lapangan [10]. *Toolset* yang tersedia mencakup Nmap, Nikto, dan Wireshark yang digunakan untuk aktivitas seperti *network mapping*, *vulnerability scanning*, *exploitation*, *sniffing*, serta analisis *traffic* jaringan.

E. Whatweb

Salah satu *tool* web *fingerprinting* yang digunakan untuk mengidentifikasi teknologi yang digunakan oleh sebuah *website* melalui analisis HTTP *response* dan konten halaman. *Tool* ini mampu mendeteksi *content management system*, *plugin*, *server software*, *scripting framework*, JavaScript *library*, *cookies*, *security header*, serta metadata umum seperti *redirect* dan *error code*. Proses deteksi dilakukan menggunakan kumpulan *plugin* yang mencocokkan *signature* pada *header* dan HTML, dengan opsi pengujian secara pasif maupun aktif serta tingkat *verbosity* yang dapat disesuaikan untuk menyeimbangkan cakupan dan tingkat *noise* [11]. Output hasil pemindaian dapat disimpan sebagai bukti sehingga hasil pengujian dapat dibandingkan antar *assessment* dan digunakan sebagai acuan untuk tahap *reconnaissance* dan *testing* selanjutnya.

F. Nikto

Nikto merupakan web *server scanner* yang berfokus pada pendeteksian *misconfiguration* umum dan kelemahan yang telah diketahui. *Tool* ini melakukan pemeriksaan terhadap file dan direktori berbahaya, *default script*, versi *server* yang sudah usang atau memiliki *vulnerability*, serta berbagai permasalahan lain yang sering ditemukan pada web *application* [12]. Nikto umumnya digunakan pada tahap awal *vulnerability analysis* untuk dengan cepat mengidentifikasi potensi risiko keamanan pada *website* target.

G. WPScan

WPScan adalah WordPress *security scanner* yang dirancang untuk menemukan kelemahan yang bersifat spesifik pada *website* berbasis WordPress. *Tool* ini melakukan *fingerprinting* terhadap versi *core* WordPress, enumerasi *plugin* dan *theme*, serta mengidentifikasi *public endpoint* seperti wp-login.php, xmlrpc.php, dan jalur REST API. Selain itu, WPScan mampu menampilkan *public users*, melakukan pemeriksaan terhadap kemungkinan kredensial lemah melalui percobaan *login* yang terkontrol, serta mencocokkan versi komponen yang terdeteksi dengan *vulnerability database* menggunakan API *token* [13].

III. METODE

A. Ruang Lingkup dan Pendekatan

Penelitian ini menggunakan pendekatan *black box* yang berfokus pada perilaku sistem yang dapat diamati dari batas luar sistem. Penelitian tidak melibatkan akses terhadap

source code, *administrative credentials*, maupun konfigurasi internal. Seluruh aktivitas pengujian hanya ditujukan pada *interface* yang dapat diakses secara publik pada *website* Pemerintah Kota XYZ dan dilaksanakan dengan mematuhi batasan hukum serta etika yang telah ditetapkan pada tahap perencanaan. Tujuan dari pendekatan ini adalah untuk memahami bagaimana sistem bereaksi terhadap *probing* eksternal yang realistis, serta menerjemahkan perilaku tersebut menjadi bukti terkait potensi kelemahan dan risiko operasional yang mungkin terjadi.

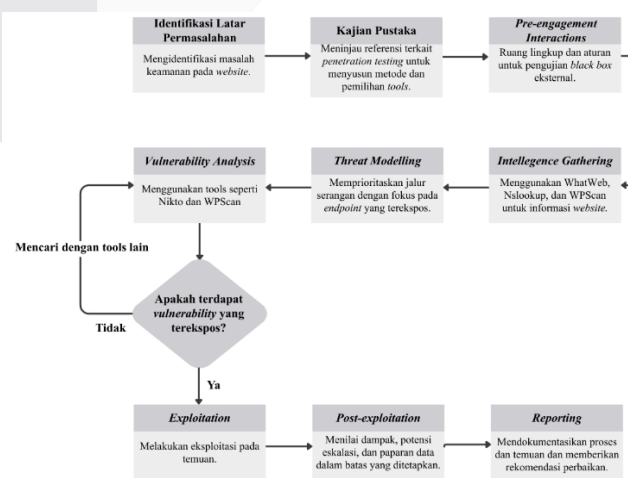
B. Lingkungan Pengujian dan Tools

Pengujian dilakukan menggunakan sebuah *workstation* Kali Linux yang telah dikonfigurasi untuk kebutuhan *security assessment*. Pada tahap *intelligence gathering*, *tool* WhatWeb digunakan untuk melakukan *fingerprinting* terhadap web *stack* yang digunakan oleh target, sedangkan nslookup digunakan untuk memverifikasi DNS *resolution* serta memperoleh informasi IP *address* dari target.

Pada tahap *vulnerability analysis*, Nikto digunakan untuk melakukan pemindaian terhadap web *server* guna mengidentifikasi file berbahaya, *outdated components*, serta *misconfigurations*. Sementara itu, WPScan difokuskan pada pengujian yang bersifat spesifik terhadap WordPress, termasuk enumerasi publik *users* melalui REST API apabila tersedia, identifikasi versi *core*, *plugin*, dan *theme*, serta pelaksanaan *controlled authentication checks* pada halaman *login*. Temuan yang dihasilkan dari kedua *tool* tersebut kemudian digunakan sebagai dasar dalam melakukan manual *review* dan menyusun *exploitation plan*.

C. Proses PTES

Pelaksanaan penelitian ini mengikuti *Penetration Testing Execution Standard* sehingga setiap aktivitas pengujian memiliki tujuan yang jelas dan urutan tahapan yang terstruktur, dimulai dari *pre-engagement interactions* dan dilanjutkan dengan *intelligence gathering*, *threat modeling*, *vulnerability analysis*, *exploitation*, *post-exploitation*, hingga *reporting*. Penerapan PTES menyediakan kesamaan bahasa dalam penentuan *scope*, alur yang sistematis dalam pengumpulan bukti, serta dasar yang konsisten dalam melakukan penilaian dan penyampaian risiko pada akhir kegiatan pengujian.



GAMBAR 1
Flow Chart Penelitian

IV. HASIL DAN PEMBAHASAN

Bagian ini berisi paparan objektif peneliti terhadap hasil-hasil penelitian berupa penjelasan dan analisis terhadap penemuan-penemuan penelitian, penjelasan serta penafsiran dari data dan hubungan yang diperoleh, serta pembuatan generalisasi dari penemuan. Apabila terdapat hipotesis, maka pada bagian ini juga menjelaskan proses pengujian hipotesis beserta hasilnya.

A. Pre-engagement Interactions

Kegiatan pengujian ini telah memperoleh izin resmi sebagai *external black box test* terhadap *website* Pemerintah Kota XYZ dengan ruang lingkup yang dibatasi pada endpoint yang dapat diakses secara publik. Penelitian ini tidak melibatkan akses terhadap *source code*, *administrative credentials*, maupun *internal network*. Eksploitasi terkontrol terhadap temuan tertentu diperbolehkan untuk mengonfirmasi dampak yang ditimbulkan, dengan tetap menjaga aspek *availability* dan *integrity* sistem. Berdasarkan kesepakatan awal dengan pemilik sistem, nama *website* serta seluruh tangkapan layar yang dapat mengungkap detail identitas target disamarkan dalam penelitian ini.

B. Intelligence Gathering

Pada tahap *intelligence gathering*, pengumpulan informasi awal dilakukan untuk memahami karakteristik dan teknologi yang digunakan oleh *website* target. Proses ini memanfaatkan *tool* WhatWeb untuk melakukan *fingerprinting* terhadap *web stack*, serta nslookup untuk memverifikasi *DNS resolution* dan memperoleh informasi *IP address* yang digunakan oleh *website*, sebagai dasar untuk tahapan pengujian selanjutnya.

TABEL 2
Hasil Intelligence Gathering

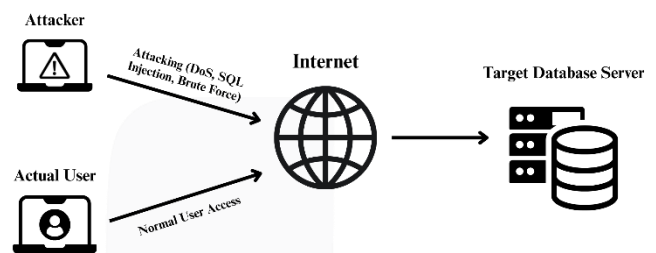
Komponen yang terdeteksi	Hasil	Keterangan singkat
Web Server	nginx	Web server yang digunakan.
Markup	HTML5	Website menggunakan standar HTML5.
Framework CSS	Bootstrap 6.7.1	Framework tampilan yang digunakan.
Library JavaScript	jQuery 3.7.1	Library JavaScript yang terdeteksi.
CMS	WordPress 6.7.1	CMS yang digunakan beserta versinya.
Page Builder	Elementor 3.26.4 (MetaGenerator)	Indikasi penggunaan plugin page builder.
Address (A record)	x.x.x.173	IP resolusi DNS.

Dari tabel 2, ditunjukkan hasil identifikasi teknologi menggunakan WhatWeb. Dari hasil tersebut, *website* terdeteksi menggunakan WordPress versi 6.7.1 dengan *plugin* Elementor versi 3.26.4 dan *library* jQuery versi 3.7.1. Pada saat pengambilan data, *web server* yang digunakan teridentifikasi sebagai nginx. Temuan ini membantu peneliti

memahami karakteristik aplikasi dan mengarahkan perhatian pada jalur standar WordPress seperti */wp-login.php* dan */wp-admin* yang umumnya menjadi titik penting dalam pengujian keamanan. Selanjutnya, nslookup menunjukkan hasil resolusi domain dan mencatat alamat IP publik dari *website* target.

C. Threat Modelling

Pada tahap *threat modelling*, dilakukan pemodelan ancaman untuk mengidentifikasi jalur serangan yang paling berpotensi dieksploitasi berdasarkan hasil *intelligence gathering*, dengan fokus pada aset utama berupa halaman publik *website*, mekanisme autentikasi WordPress, serta komponen aplikasi web yang memproses input pengguna. Permukaan serangan diprioritaskan pada layanan HTTP dan HTTPS serta *endpoint* khas WordPress seperti halaman *login*, dengan mempertimbangkan potensi kerentanan yang umum muncul, antara lain SQL Injection, *brute force*, *clickjacking* akibat lemahnya pengaturan *security header*, serta kesalahan konfigurasi *server* atau *plugin*. Hasil pemodelan ini digunakan sebagai dasar penentuan fokus pengujian pada tahap *vulnerability analysis* dan *exploitation* agar pengujian diarahkan pada skenario yang paling realistis dan berdampak signifikan terhadap kerahasiaan, integritas, dan ketersediaan sistem.



GAMBAR 2
Threat Modelling

D. Vulnerability Analysis

Pada tahap *vulnerability analysis*, proses pengujian dilakukan untuk mengidentifikasi kelemahan konfigurasi dan potensi kerentanan pada *website* target menggunakan *tool* Nikto dan WPScan. Nikto digunakan untuk mendeteksi *misconfiguration* pada *web server* serta keberadaan komponen yang berisiko, sedangkan WPScan difokuskan pada pengujian yang bersifat spesifik terhadap WordPress, termasuk identifikasi *endpoint* standar dan mekanisme proteksi yang diterapkan.

Hasil pengujian menunjukkan adanya beberapa indikator kelemahan konfigurasi *server*. Dari seluruh temuan Nikto, penelitian ini memfokuskan pembahasan pada tiga temuan utama yang paling relevan untuk ditindaklanjuti, yaitu tidak diterapkannya *header* keamanan X-Frame-Options dan X-Content-Type-Options yang berpotensi membuka peluang terjadinya *clickjacking* serta risiko MIME sniffing, serta terkonfirmasinya penggunaan WordPress beserta kemunculan *endpoint* standar seperti halaman *login* yang perlu divalidasi lebih lanjut. Pada sisi lain, proses pemindaian menggunakan WPScan tidak dapat diselesaikan dan berakhir dengan pesan *timeout*, sementara akses *browsing* normal tetap berjalan. Kondisi ini ditafsirkan sebagai indikasi adanya mekanisme penyaringan pada lapisan depan *server*, seperti *Web Application Firewall* atau sistem pencegah intrusi, yang

membatasi permintaan otomatis berulang dan menunjukkan keberadaan lapisan proteksi tambahan pada *website*.

E. *Exploitation*

Pada tahap *exploitation*, pengujian dilakukan secara bertahap sesuai urutan yang telah direncanakan. Pengujian meliputi percobaan SQL Injection pada *endpoint login* dan parameter input, uji *denial of service* untuk mengamati ketahanan serta mekanisme rate limiting, pengujian *brute force* secara terkontrol pada halaman *login* untuk mengevaluasi penguatan autentikasi, serta pengujian *clickjacking* menggunakan halaman *iframe* guna memverifikasi penerapan UI *protection header* dan potensi pemicu aksi di dalam *frame*.

1. SQL Injection

Langkah awal pengujian SQL injection dilakukan dengan cara mencoba *login* yang valid melalui *browser* pada *endpoint* /wp-login.php, lalu seluruh *request* HTTP POST diekspor ke file teks dan dijalankan kembali menggunakan opsi -r pada SQLMap. Parameter yang digunakan --level=1 dan --risk=1 agar *payload* tetap konservatif dan sesuai batas pengujian *black box* yang sudah ditetapkan. Level 1 membatasi jumlah pengecekan dan parameter yang diuji, sedangkan risk 1 menghindari *payload* yang lebih berat dan berpotensi lebih intrusif.

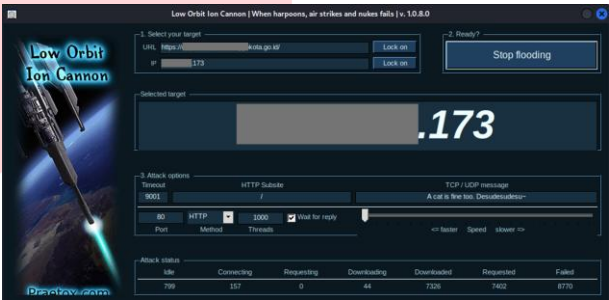
TABEL 3
Hasil Tools Sqlmap

No	Output	Keterangan
1	POST parameter log	Parameter yang diuji oleh sqlmap pada <i>request login</i> .
2	POST parameter 'log' does not seem to be injectable	Tidak ditemukan indikasi SQL Injection pada parameter yang diuji.
3	all tested parameters do not appear to be injectable	Tidak ada parameter yang teridentifikasi dapat dieksploitasi.
4	If you suspect ... protection mechanism involved (e.g. WAF)	Terdapat indikasi kemungkinan mekanisme proteksi yang memengaruhi hasil uji, sehingga hasil dipahami sebagai tidak terkonfirmasi pada skenario ini.

Berdasarkan tabel 3, hasil mengindikasikan parameter yang diuji tidak dapat diinjeksi dan proses berakhir dengan catatan bahwa *parameter log* kemungkinan memang tidak dapat diinjeksi. *Output* juga menyarankan peningkatan --level atau --risk, atau penggunaan opsi *tamper* bila dicurigai ada mekanisme perlindungan. Jika dikaitkan dengan temuan sebelumnya berupa *timeout* saat *probe* otomatis, kondisi ini konsisten dengan adanya WAF atau *filtering* di lapisan depan *server* yang membatasi atau mengubah lalu lintas dari *scanner*, sehingga pada batas pengujian saat ini upaya SQL injection dicatat sebagai belum terkonfirmasi.

2. *Denial of Service* (DoS)

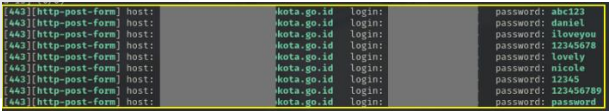
Pengujian *denial of service* dilakukan secara terkendali menggunakan HTTP *flood* dengan *tool* LOIC terhadap *hostname* dan IP target untuk mengamati ketahanan layanan serta mekanisme *rate limiting* dan *filtering*. Hasil pengujian menunjukkan bahwa *website* sempat tidak dapat diakses dari sisi perangkat penguji, namun tetap dapat diakses secara normal melalui perangkat lain atau jalur akses berbeda, yang mengindikasikan adanya mitigasi berbasis sumber serangan. Kondisi ini diduga disebabkan oleh penerapan WAF, CDN, atau *reverse proxy* yang melakukan pemblokiran atau pembatasan *traffic* berdasarkan IP, sehingga pengujian dicatat sebagai mekanisme mitigasi yang efektif dan bukan sebagai keberhasilan serangan DoS terhadap layanan secara keseluruhan.



GAMBAR 3
Tools LOIC

3. *Brute Force Login*

Pengujian *brute force* dilakukan untuk mengevaluasi kekuatan mekanisme autentikasi pada *website* berbasis WordPress dengan menargetkan *endpoint* wp-login.php menggunakan kecepatan yang dibuat konservatif agar ketersediaan layanan tetap terjaga. *Username* diperoleh dari informasi publik yang tersedia melalui *endpoint* REST wp-json/wp/v2/users, kemudian dipasang dengan daftar kata sandi umum dan diuji menggunakan *tool* Hydra dengan modul http-form-post, karena WPScan terindikasi terblokir dan metode *login* berbasis XML-RPC tidak tersedia atau terfilter. Pengujian ini bertujuan untuk menilai kemungkinan keberhasilan *brute force* dalam kondisi realistis tanpa menimbulkan gangguan terhadap layanan.



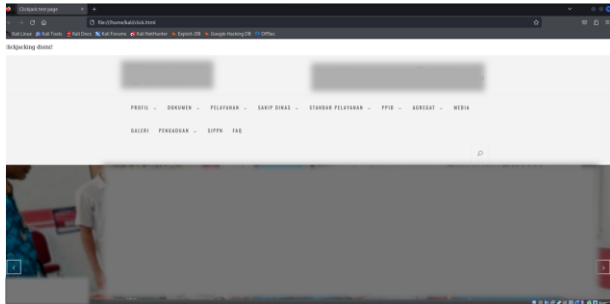
GAMBAR 4
Hasil Hydra

Seperti terlihat pada gambar 4, hasil pengujian menunjukkan bahwa daftar kata sandi kandidat yang diuji cukup beragam, misalnya pola kata sandi umum seperti “abc123”, “iloveyou”, “12345678”, dan variasi serupa. Akan tetapi, seluruh respons yang diterima menunjukkan kredensial tidak valid dan tidak ada sesi yang berhasil diperoleh. Selain itu, halaman *login* menampilkan pola kegagalan yang sama, namun pada beberapa momen terlihat adanya *timeout* sesekali dan respons yang melambat setelah sejumlah percobaan. Pola ini konsisten dengan adanya kontrol keamanan seperti WAF atau IPS

yang menerapkan pembatasan percobaan dan memfilter pengiriman *form* otomatis, sehingga upaya *brute force* dicatat tidak berhasil dalam batas pengujian yang dilakukan

4. Clickjacking

Pengujian *clickjacking* dilakukan dengan membuat sebuah halaman HTML sederhana yang memuat *website* target di dalam elemen `<iframe>` dengan ukuran penuh, sesuai tampilan pada gambar hasil pengujian. Halaman ini digunakan sebagai media uji untuk memastikan apakah *website* dapat ditampilkan di dalam *frame* dan apakah elemen antarmuka seperti menu atau tombol masih dapat diklik ketika berada di dalam *frame*.



Gambar 5
Hasil Clickjacking

Berdasarkan gambar 5, *website* berhasil dirender di dalam *frame* dan menu tetap dapat digunakan secara normal. Kondisi ini mengindikasikan bahwa kontrol perlindungan *anti-framing* seperti X-Frame-Options atau kebijakan Content Security Policy pada aturan *frame-ancestors* tidak ada, tidak aktif, atau tidak efektif. Apabila kondisi ini dibiarkan, penyerang berpotensi menumpuk lapisan transparan di atas *website* untuk mengelabui pengguna agar mengklik elemen tersembunyi, memanfaatkan klik tersebut untuk skenario *phishing*, atau menggabungkannya dengan teknik lain seperti CSRF agar aksi tertentu dapat terpanggil ketika pengguna masih memiliki sesi yang valid.

F. Post-exploitation

Tahap *post-exploitation* pada penelitian ini dilakukan secara terbatas untuk memverifikasi dampak yang realistis tanpa menimbulkan kerusakan pada sistem maupun mengganggu ketersediaan layanan publik, sesuai dengan ruang lingkup pengujian *black box* yang hanya memanfaatkan endpoint publik. Percobaan SQL Injection tidak terkonfirmasi, pengujian *denial of service* hanya memicu mitigasi berbasis sumber tanpa menyebabkan gangguan menyeluruh, dan percobaan *brute force* tidak menghasilkan sesi tidak sah, sehingga tidak terdapat langkah lanjutan yang dapat dilakukan dari ketiga vektor serangan tersebut.

Temuan paling signifikan pada tahap ini adalah keberhasilan *clickjacking*, di mana *website* dapat dimuat di dalam *iframe* dan elemen antarmuka tetap dapat diinteraksikan, yang mengindikasikan kontrol *anti-framing* belum diterapkan atau tidak berjalan efektif. Kondisi ini divalidasi secara non-destruktif melalui interaksi ringan, dan

dalam skenario serangan nyata dapat dimanfaatkan untuk mengelabui pengguna agar memicu aksi tertentu tanpa disadari, terutama ketika pengguna berada dalam sesi yang valid. Oleh karena itu, diperlukan penerapan kontrol tambahan seperti mekanisme *anti-framing*, token CSRF, pembatasan aksi sensitif, serta dialog konfirmasi pada tindakan penting untuk meminimalkan risiko penyalahgunaan interaksi pengguna.

G. Reporting

Berdasarkan hasil pengujian, percobaan SQL Injection, *denial of service*, dan *brute force* tidak berhasil dieksploitasi, yang mengindikasikan adanya mekanisme proteksi seperti *filtering* lalu lintas, *rate limiting*, serta kontrol autentikasi yang berjalan pada *website*. Meskipun demikian, temuan yang paling signifikan adalah keberhasilan *clickjacking*, yang ditunjukkan oleh kemampuan *website* untuk dimuat di dalam *iframe* dan tetap memungkinkan interaksi antarmuka, sehingga mengindikasikan tidak diterapkannya atau tidak efektifnya *header anti-framing*. Kondisi ini berpotensi memengaruhi interaksi pengguna, khususnya pada halaman sensitif, sehingga rekomendasi prioritas tinggi adalah penerapan X-Frame-Options dengan kebijakan *deny* atau Content Security Policy melalui aturan *frame-ancestors*, disertai rekomendasi lanjutan berupa pembatasan enumerasi pengguna melalui REST API, peninjauan eksposur XML-RPC, serta pemutakhiran *core CMS* dan *plugin* secara berkala.

V. KESIMPULAN

Pengujian keamanan pada *website* Pemerintah Kota XYZ dilakukan menggunakan pendekatan *black box* dengan alur *Penetration Testing Execution Standard* (PTES), sehingga evaluasi difokuskan pada endpoint publik dari perspektif pihak eksternal. Tahapan pengujian meliputi *intelligence gathering* untuk mengidentifikasi teknologi dan permukaan serangan, *threat modeling* untuk menentukan fokus ancaman, *vulnerability analysis* untuk mengamati indikasi kelemahan dan mekanisme proteksi, *exploitation* yang dilakukan secara terukur, serta *post-exploitation* terbatas untuk menilai dampak temuan tanpa mengubah sistem, dengan seluruh hasil dirangkum pada tahap *reporting*.

Hasil analisis menunjukkan bahwa sebagian besar skenario pengujian tidak menghasilkan eksploitasi yang berhasil. Percobaan SQL Injection tidak terkonfirmasi, pengujian *denial of service* hanya memicu mitigasi berbasis sumber tanpa mengganggu ketersediaan layanan secara menyeluruh, dan percobaan *brute force* tidak menghasilkan sesi tidak sah serta menunjukkan adanya pembatasan yang selaras dengan mekanisme proteksi seperti WAF atau IPS. Namun demikian, ditemukan satu kelemahan yang signifikan, yaitu *clickjacking*, di mana *website* dapat dimuat di dalam *iframe* dan elemen antarmuka tetap interaktif, yang mengindikasikan kontrol *anti-framing* belum diterapkan atau tidak berjalan efektif.

Berdasarkan temuan tersebut, rekomendasi perbaikan difokuskan pada penerapan kontrol *anti-framing* sebagai prioritas utama untuk mencegah *clickjacking*, melalui penggunaan header X-Frame-Options atau Content Security Policy dengan aturan *frame-ancestors* yang sesuai.

Rekomendasi lanjutan mencakup peninjauan eksposur informasi publik yang tidak diperlukan, pembatasan kemungkinan enumerasi pengguna melalui REST API, serta pemeliharaan pembaruan *core* CMS dan *plugin* secara berkala. Pengujian ulang dengan pendekatan *black box* yang sama disarankan setelah perbaikan dilakukan guna mengevaluasi efektivitas mitigasi secara konsisten.

REFERENSI

- [1] S. Mynenko and O. Lyulyov, "The Impact of Digitalization on the Transparency of Public Authorities," *Bus. Ethics Leadersh.*, vol. 6, no. 2, pp. 103–115, 2022, doi: 10.21272/bel.6(2).103-115.2022.
- [2] . D., W. Jeberson, and K. Jeberson, "Enhancing Web Security: Implementing CAPTCHA for Government Websites," *Int. J. Innov. Sci. Res. Technol.*, vol. 9, no. 3, pp. 1281–1287, 2024, doi: 10.38124/ijisrt/ijisrt24mar1463.
- [3] H. Gawel, "Hacktivism," vol. 13, 2024.
- [4] S. N. Anam, D. Suhartono, and A. Pramono, "Managing Information Security Risks in Detecting, Handling, and Preventing Cybersecurity Incidents on Local Government Websites," *J. Teknol. Sist. Inf. dan Apl.*, vol. 7, no. 4, pp. 1512–1520, 2024, doi: 10.32493/jtsi.v7i4.44099.
- [5] W. Zhang, J. Xing, and X. Li, "Penetration Testing for System Security: Methods and Practical Approaches," vol. 1, no. 1, pp. 1–17, 2025, [Online]. Available: <http://arxiv.org/abs/2505.19174>
- [6] P. Lachkov, L. Tawalbeh, and S. Bhatt, "Vulnerability Assessment for Applications Security Through Penetration Simulation and Testing," *J. Web Eng.*, vol. 21, no. 7, pp. 2187–2208, Dec. 2022, doi: 10.13052/JWE1540-9589.2178.
- [7] D. Priyawati, S. Rokhmah, and I. C. Utomo, "Website Vulnerability Testing and Analysis of Internet Management Information System Using OWASP," *Int. J. Comput. Inf. Syst. Peer Rev. J.*, vol. 3, no. 3, pp. 143–147, 2022, doi: 10.29040/ijcis.v3i3.90.
- [8] M. F. Safitra, M. Lubis, and A. Widjajarto, "Security Vulnerability Analysis using Penetration Testing Execution Standard (PTES): Case Study of Government's Website," *ACM Int. Conf. Proceeding Ser.*, no. 3, pp. 139–145, 2023, doi: 10.1145/3592307.3592329.
- [9] S. V. Thakare and S. L. Parab, "Network Security in IT Infrastructure," pp. 261–264, 2023, doi: 10.48175/IJARST-12136.
- [10] K. L. Pengertian, "Sejarah Kali OS," pp. 1–8, 2023.
- [11] A. Kar, A. Natadze, E. Branca, and N. Stakhanova, "HTTPFuzz: Web Server Fingerprinting with HTTP Request Fuzzing," *Proc. Int. Conf. Secur. Cryptogr.*, vol. 1, no. January, pp. 261–271, 2022, doi: 10.5220/0011328900003283.
- [12] R. S. Devi and M. M. Kumar, "Testing for Security Weakness of Web Applications using Ethical Hacking," *Proc. 4th Int. Conf. Trends Electron. Informatics, ICOEI 2020*, no. September, pp. 354–361, 2020, doi: 10.1109/ICOEI48184.2020.9143018.
- [13] D. Reti, K. Elzer, and H. D. Schotten, "SCANTRAP: Protecting Content Management Systems from Vulnerability Scanners with Cyber Deception and Obfuscation," *Int. Conf. Inf. Syst. Secur. Priv.*, pp. 485–492, 2023, doi: 10.5220/0011667400003405.